

Unit tests 01 - Introduction

Zdenko Pavlik

Scheidt&Bachmann Slovakia s.r.o.

Unit tests 01 - Introduction

... done with support of



Figure 1: scheidt&bachmann

Agenda

- ▶ Why tests are important
- ▶ Different types of testing
- ▶ Importance of unit testing
- ▶ How to write unit tests
- ▶ Best practices for unit testing

Why tests are important

- ▶ Risk Mitigation
 - ▶ Testing helps identify defects and failures early in the development process when they are less expensive to fix. This reduces project risks related to quality, security, performance, etc.
- ▶ Confidence
 - ▶ Executing a well-planned software test strategy provides confidence that the software works as intended before its release.
- ▶ Compliance
 - ▶ Testing can ensure that software adheres to standards, regulations, and compliance requirements. This is especially critical for safety-critical systems.
- ▶ User Satisfaction
 - ▶ Rigorous testing from a user perspective can verify usability, functionality, and compatibility. This increases customer/user satisfaction and reduces negative impacts to an organization's reputation or finances from poor quality products.

Why tests are important

- ▶ Optimization
 - ▶ Testing provides vital feedback that can be used to continuously improve software quality, user experience, security, performance and other product attributes.
- ▶ Cost Savings
 - ▶ Investing in testing activities reduces downstream costs related to defects found post-release. It is much cheaper to find and fix bugs earlier in the development cycle.

... and many more reasons

Why tests are important (SW developer perspective)

- ▶ Enable us testing corner case scenarios
 - ▶ Testing enables us to validate that our code behaves correctly under various conditions, including edge cases.
- ▶ Ensure code stability
 - ▶ Testing helps maintain code stability by catching regressions and unintended side effects early in the development process.
- ▶ Facilitate refactoring and code maintenance
 - ▶ Testing allows developers to confidently refactor and maintain code, knowing that existing functionality is protected by a suite of automated tests.

Why tests are important (SW developer perspective)

- ▶ They are instant!
 - ▶ Unit tests can be executed quickly (often in seconds or minutes), providing immediate feedback to developers about the correctness of their code.
- ▶ **Disclaimer**
 - ▶ They took time to write and maintain. Often it is 2x time of development. But AI can help significantly!
 - ▶ Unit tests are not a silver bullet; they complement other testing types and practices to ensure overall software quality.

Different types of (automated) testing

Ordered from least complex to most complex.

Different types of (automated) testing

Unit tests

Written by: Developers

Target: Classes

Purpose: Test individual units of code in isolation

Complexity: Low

Example: Test files that are part of repository, i.e. `EventHandlerTest.cpp`

Unit tests are very low level and close to the source of an application. They consist in **testing individual methods and functions** of the classes, components, or modules used by your software. Unit tests are generally quite cheap to automate and can run very quickly by a continuous integration server.

Different types of (automated) testing

Smoke testing

Written by: Developers or QA engineers

Target: Application

Purpose: Check the basic functionality of an application

Complexity: Low

Smoke tests are basic tests that check the **basic functionality** of an application. They are meant to be quick to execute, and their goal is to give you the assurance that the major features of your system are working as expected. Smoke tests can be useful right after a new build is made to decide whether or not you can run more expensive tests, or right after a deployment to make sure that the application is running properly in the newly deployed environment.

Fun fact: Naming comes from hardware testing, where a “smoke test” was a quick check to see if a device would power on without emitting smoke.

Different types of (automated) testing

Fuzz testing

Written by: Developers or QA engineers

Target: Application or Classes

Purpose: Identify security vulnerabilities and robustness issues by providing invalid, unexpected, or random data as inputs

Complexity: Medium

Fuzz testing, or fuzzing, is a type of automated testing that involves **providing invalid, unexpected, or random data as inputs** to a computer program. The program is then monitored for crashes, memory leaks, or other unexpected behaviors. Fuzz testing is particularly useful for identifying security vulnerabilities and robustness issues in software.

Different types of (automated) testing

Integration tests

Written by: Developers

Target: Modules or services

Purpose: Test interactions between different modules or services

Complexity: Medium

Integration tests verify that different **modules or services** used by your application work well together. For example, it can be testing the interaction with the database or making sure that microservices work together as expected. These types of tests are more expensive to run as they require multiple parts of the application to be up and running.

Different types of (automated) testing

Functional tests

Written by: Developers or QA engineers

Target: Application

Purpose: Test the application against business requirements

Complexity: Medium

Functional tests focus on the **business requirements** of an application. They only verify the output of an action and do not check the intermediate states of the system when performing that action. There is sometimes a confusion between integration tests and functional tests as they both require multiple components to interact with each other. The difference is that an integration test may simply verify that you can query the database while a functional test would expect to get a specific value from the database as defined by the product requirements.

Different types of (automated) testing

End-to-end tests

Written by: Developers or QA engineers

Target: Application

Purpose: Test the application from start to finish

Complexity: High

End-to-end testing replicates a **user behavior** with the software in a complete application environment. It verifies that various user flows work as expected and can be as simple as loading a web page or logging in or much more complex scenarios verifying email notifications, online payments, etc... End-to-end tests are very useful, but they're expensive to perform and can be hard to maintain when they're automated. It is recommended to have a few key end-to-end tests and rely more on lower level types of testing (unit and integration tests) to be able to quickly identify breaking changes.

Different types of (automated) testing

Acceptance testing

Written by: Developers or QA engineers

Target: Application

Purpose: Verify if a system satisfies business requirements

Complexity: High

Acceptance tests are formal tests that verify if a system satisfies **business requirements**. They require the entire application to be running while testing and focus on replicating user behaviors. But they can also go further and measure the performance of the system and reject changes if certain goals are not met.

Different types of (automated) testing

Performance testing

Written by: Developers or QA engineers

Target: Application

Purpose: Evaluate how a system performs under a particular workload

Complexity: Medium

Performance tests evaluate how a system performs under a **particular workload**. These tests help to measure the reliability, speed, scalability, and responsiveness of an application. For instance, a performance test can observe response times when executing a high number of requests, or determine how a system behaves with a significant amount of data. It can determine if an application meets performance requirements, locate bottlenecks, measure stability during peak traffic, and more.

Different types of (automated) testing

Sources:

<https://www.atlassian.com/continuous-delivery/software-testing/types-of-software-testing>

<https://www.geeksforgeeks.org/software-testing/types-software-testing/>

... enough of theory

Let's practice unit tests.

What are unit tests?

- ▶ Isolated test cases (functions) validating classes, methods and functions.
- ▶ Quick to execute (seconds).
- ▶ Intended to use regularly (at least before merging to develop branch).
- ▶ Advantage is that we can test edge cases in a simple way.
- ▶ Isolated from production code, usually located in `test` directory.

Unit testing

I will be using GoogleTest for unit testing in this tutorial.

<https://github.com/google/googletest>

<https://google.github.io/googletest/primer.html>

<https://google.github.io/googletest/>

Example

PersonSalaryCalculatorTest.cpp

```
TEST(PersonSalaryCalculatorTest, simpleCreation)
```

```
{
```

```
    PersonSalaryCalculator person("Bob", 2000);
```

```
    EXPECT_EQ(person.getName(), "Bob"); // Case sensitive, simple to use
```

```
}
```

```
TEST(PersonSalaryCalculatorTest, simpleSalary)
```

```
{
```

```
    PersonSalaryCalculator person1("Bob", 1000);
```

```
    EXPECT_EQ(person1.calculateFinalSalary(), 750);
```

```
}
```

Additional macros

```
EXPECT_EQ(1, 1);  
EXPECT_TRUE(1+1 == 2);
```

```
EXPECT_GT((1+1), 1); // Greater than  
EXPECT_LT((1+1), 6); // Less than
```

```
EXPECT_STREQ("One", "One");  
EXPECT_STRCASEEQ("One", "ONE");
```

```
EXPECT_ANY_THROW({ throw std::exception(); });  
EXPECT_DEATH(segfault(), "Memory access violation");  
EXPECT_THROW(throwsException(), std::runtime_error);  
EXPECT_ANY_THROW(throwsException());  
EXPECT_NO_THROW(noThrow());
```

EXPECT vs ASSERT

```
EXPECT_EQ(1, 1);
```

vs.

```
ASSERT_EQ(1, 1);
```

ASSERT_* macros are similar to EXPECT_* macros, but they will **abort** the current test if the assertion fails.

Demo

- ▶ integrating unit tests into project (and executing)
- ▶ writing unit tests
- ▶ using AI to write unit tests for us
- ▶ coverage report

Q&A

